

AI Engineer — Take-Home Exercise

This is the exercise for the AI Engineer role. It's meant to feel like a real version of the work, not a generic take-home.

A few ground rules first.

Deadline. You have 24 hours from when you receive this to submit. That's a hard cutoff, not a suggestion. It's there so the time-box below actually means something instead of turning into unlimited days of polish spread out over a week.

Time-box. We suggest not spending more than about 4 hours total. That's a suggestion, not a hard cap: if you want to spend more within the 24 hours, that's your call, entirely optional.

What we're evaluating. Three things, specifically. Whether you can frame the actual impact of what you're building, in terms someone outside engineering would care about, not just "the model scores accounts." Whether you can actually build an agent that does something real with a model's output, not describe one. And whether you think about how you'd know it's still working after you stop watching it, not just whether it works today. Code polish beyond what those three things need isn't the point here, and neither is auditing the model to research-grade rigor, that's a different role's job. This one is about shipping something real and being honest about what could make it wrong.

Using AI. We'd genuinely like you to use it, this is how we work day to day, and an exercise that pretended otherwise wouldn't tell us much. We'd actually encourage assisted coding tools specifically, Claude Code, Codex, Cursor, Antigravity, or anything similar, alongside whatever chat tool, agent, or other AI-supported workflow you're comfortable with. One practical note: Dialpad doesn't provide an AI tool, account, subscription, or managed environment for this exercise, so you'll be working with whatever you already have access to. What we care about isn't whether you used AI, it's how: how you break down the problem, how you prompt, how you verify what comes back, and whether you can explain and modify the result yourself. Two things we ask in return: log specific sessions and prompts in `RESEARCH-LOG.md` as you go, not a summary written afterward, and tell us about one place where you corrected or overrode it, somewhere it gave you something wrong or generic, and what you changed. Be ready to extend or defend any part of the repo live, on your own, regardless of what AI tooling helped you draft. If you use AI live during the follow-up presentation, share your screen so we can see the full interaction: prompts, outputs, and iterations.

We want your real working history, not a cleaned-up final state. Commit as you go, real and incremental, not one commit at the end. We read the commit history as part of understanding how you worked, alongside the research log. When you're done, push to a public git repo and send us the link, that's the whole submission.

A starter repo is attached: a README, pinned dependencies, an already-trained model, and the data you'll use. If something is genuinely ambiguous, use your judgment and state your assumption. Some of the ambiguity here is intentional.

The scenario

Cordilla Systems is a roughly 900-person B2B workflow software company. Sales runs on Salesforce.

On top of a few thousand paying accounts, the instance holds tens of thousands of non-customers: Prospects, Suspects, and Former Customers, mostly untouched.

Here's how this exercise actually starts. The VP of Sales comes to you with an ask:

"Do something with all this account data so reps stop guessing which accounts to call."

That sounds like a scoped request. It isn't one. Nobody has said what it's supposed to optimize for, what decision it changes, what shape it should take, or what it's actually worth if it works. You've been handed a vague mandate, not a spec, which happens constantly in this job.

While looking into what Cordilla actually has to work with, you find something nobody mentioned when they handed you this brief: a model, already built, already trained, sitting unused. It's attached to this exercise as `model/model.pkl`, along with the data it was trained on and a fresh batch to run it against. Nobody's vouching for it, and nobody's asked you to rebuild or re-validate it to research standards, that's not this job. What is this job: decide roughly what it's worth if this works, build the thing that actually puts its output to work, and think honestly about how you'd know if it stopped being right after you stop watching it.

One thing worth knowing going in: an earlier scoring effort at Cordilla looked promising in testing, strong enough that there was real excitement about shipping it, and then quietly lost credibility a couple of quarters after launch once the scores stopped matching what reps were seeing in the field. Nobody ever fully wrote up why, and nobody was watching closely enough to catch it while it was happening. That's the shape of the job this exercise is testing for.

What you're designing

Three things. We're deliberately not telling you exactly what each one should contain, we want to see what you think matters, not fit a checklist we already had in mind. A hint at the minimum bar for each, not a spec.

1. Impact framing. Before you build anything, frame what this is actually worth. Who's making a decision based on this model's output, what's the decision, and roughly what does it change if it works, versus if it's wrong in each direction. This isn't a data-science exercise, we're not asking you to validate the model to research rigor, we're asking whether you can turn "there's a model sitting here" into a real, defensible read on impact that a VP of Sales would actually believe. Ground it in the data you have, don't just assert a number.

2. Build an agent. Given the model's output on `data/accounts_to_score.csv`, design and build a real, working agent that does something with that output, something that changes what an SDR or account manager actually does with their day. We're not telling you what shape this takes. A hint, not a checklist, at the minimum we'd want to see something real on: what tools or actions it has available to it and why those, the overall structure and control flow, which agent framework you'd reach for and why, or why you'd skip one entirely. It needs to actually run.

3. Monitoring. This is the part most take-homes skip, and it's the part that actually matters once something like this is running unattended for months. How would you, or anyone, know if this stopped working? Not "add logging" as a sentence, an actual design: what specifically would you watch, what would a real problem look like in that signal versus normal noise, and what would you do about it. Think about failure modes that don't crash anything, the kind that just quietly produce wrong output while everything looks fine, that's the failure mode that killed the earlier scoring effort at Cordilla. We want at least one piece of this made concrete enough that we can tell whether you could actually build it, not just name it.

What's in the box

`model/model.pkl` is a trained scikit-learn pipeline that scores an account's likelihood of converting. It's already fit and ready to call `.predict_proba()` on. Don't retrain it.

`data/training_data.csv` is the historical, labeled data this model was actually trained on, 1,200 accounts. You don't need to audit this to research rigor, but it's real data and worth actually looking at, not just skimming, if something about it changes how much you'd trust the model, that's fair game for your impact framing and your monitoring design.

`data/accounts_to_score.csv` has 300 accounts, unlabeled, for you to run the model against as part of the agent step. Both files are static snapshots generated as of 2026-08-01. Treat that date as “today” for any recency/age calculation, not your system clock.

Columns in both files: `account_id`, `account_type` (Prospect, Suspect, or Former Customer), `snapshot_date`, `employee_count`, `industry`, `intent_score` (missing on about 40% of rows, mirroring partial vendor coverage), `mql_count_90d`, `trial_started`, `trial_active_users`, `web_touchpoints_90d`, `sales_contacts_90d`, and `converted_within_90d` (the target, training file only).

The real picture is bigger than these two CSVs. In practice this signal would come from Salesforce, a marketing automation platform, a third-party intent data vendor, a firmographic and technographic enrichment vendor, a web and ad attribution vendor, and product usage telemetry, each with real coverage gaps. Intent data skews toward larger accounts. Product usage only exists for accounts that started a trial. Historical outreach-to-conversion rates are well under 1% for cold accounts and low single digits for anything with recent engagement. Reason about that full picture in your impact framing even though you’re only hands-on with the two CSVs.

We’re not providing an LLM API key or any managed environment for this exercise. If your agent design calls a model as part of its reasoning or tool use, mock that call: a clearly designed stand-in with an obvious, documented spot where a real call would plug in (what prompt or instruction, what inputs, what tools it could invoke, what it would return) is judged the same as a live call would be. We’re evaluating the design and the structure around it, not whether a network request actually fires.

Deliverables

You’re building a small git repo. A starter scaffold is attached, use it or restructure it, your call, as long as everything below ends up in the repo.

1. The agent (`agent/`). A rough, real, working build. It needs to load the model, do something with its output on `data/accounts_to_score.csv`, and actually run end to end, mocked model/tool calls where noted above are fine. It doesn’t need to handle every edge case, and it doesn’t need to be production-grade code, but it needs to be real: something we could run, read, and ask you to change live.

2. Monitoring (`monitoring/` or folded into `agent/` , your call). At least one real, concrete piece of this: a health check, a data-quality assertion, a drift signal, an alert

condition, whatever you think actually matters here, specific enough that we can tell whether you could actually write it, not just that you know monitoring should exist. Code, pseudocode, or a precisely specified check are all fine, a one-line “add logging” is not.

3. Written proposal (`PROPOSAL.md` , roughly 800 to 1,200 words) covering all three areas above: the impact framing (with real numbers grounded in the data, not asserted), the agent’s design choices (tools, structure, framework and why, deployment), and the monitoring design (what you’d watch, what a real problem looks like versus noise, what happens when it trips).

4. Research log (`RESEARCH-LOG.md`), kept as you go, not written after the fact. Hypotheses, what looking at the data told you, dead ends, and specifically what you asked your AI tool and what came back. **Your last entry in this file should pull together the base text, numbers, hypotheses, and assumptions you’d actually use to present this**, not the presentation itself, just the raw material for it in one place, so it’s clear what you’d stand behind in the room.

5. Real git history. Commit as you actually progress. Don’t squash into one commit at the end.

Suggested repo structure

```
cordilla-account-scoring/
├── README.md           how to set up and run your code
├── requirements.txt
├── model/
│   └── model.pkl       provided, don't retrain
├── data/
│   ├── training_data.csv    provided, don't modify or regenerate
│   └── accounts_to_score.csv provided, don't modify or regenerate
├── agent/
│   └── ...              your agent
├── monitoring/
│   └── ...              your monitoring design/checks (or folded into
agent/)
├── PROPOSAL.md
└── RESEARCH-LOG.md
```

Keep it light. No formal test suite, no packaging, no CI/CD. It’s a working prototype, not a production service.

Submitting

Push this to a public git repo and send us the link when you're done. That's the submission, nothing else to package or attach.

Once we have it, we'll schedule the presentation as a separate follow-up meeting, not something you need to prepare before submitting.

Things you don't need to do

- Audit the model to research-grade rigor, cross-validate it, or write a formal statistical case for or against trusting it. Look at the data enough to ground your impact framing and monitoring design, that's the bar.
- Retrain, tune, or improve the model.
- Get a real LLM API call working. A well-documented mock is judged the same, see above.
- Build a full deployment pipeline. Describe how you'd do it and why in `PROPOSAL.md` ; a rough sketch or config stub in the repo is welcome but not required.
- Handle every data quality edge case. Flag the ones that matter to your impact framing or monitoring design and move on.
- Draw a formal architecture diagram, unless it's genuinely faster for you than writing it out.
- Prepare a presentation before you submit. That comes after, see below.

The presentation

This happens after you submit, once we've scheduled the follow-up meeting. Don't build it beforehand.

20 to 25 minutes with a small panel: plan for 10 to 12 minutes walking through your proposal and a live run of your agent, and the rest as discussion. The panel will extend the scenario live with a new constraint, a resourcing limit, a data source disappearing, a stakeholder objection, and wants to see how your thinking, and your code, hold up.

Use Google Slides, Figma, or whatever tool you're comfortable with, or just open the repo, your call. There's no required format. But how you visually or verbally tell this story is genuinely part of what we're looking at, not an afterthought to the repo. A rep or a VP of Sales isn't going to read your `PROPOSAL.md` , so showing us you can translate real findings into something a non-technical audience would actually follow and remember carries real weight here.

This is about your reasoning and your code, not a formal code review, but come ready to open the repo and run it live if asked.

Looking forward to seeing what you build.